

מדריך למימוש של פיצ'ר חישוב הנקודות

מטרת הפיצ'ר

פיצ'ר ניקוד המשחק אחראי על מעקב אחר ביצועי המשתמש, ניהול סיום המשחק, והצגת תוצאות הסיום. הפיצ'ר עושה שימוש בארבעה Counters כדי לעקוב אחר תשובות נכונות/שגויות ולקבוע מתי המשחק הסתיים.

מבני הנתונים המרכזיים

מערכת הניקוד עושה שימוש בארבעה משתנים המאוחסנים במחלקת Game:

```
class Game {  
  
private:  
  
    int correct_answers_m;    // Increments when user answers correctly  
  
    int wrong_answers_m;      // Increments when user answers wrong OR times out  
  
    int images_completed_m;    // Tracks how many images have been shown  
  
    bool game_loop_complete_m; // Flag when all images completed  
  
};
```

אתחול:

```
Game(lv_obj_t *img, lv_obj_t *timer) :  
  
    correct_answers_m(0), wrong_answers_m(0),  
  
    images_completed_m(0), game_loop_complete_m(false) {  
  
    // Game starts fresh with all counters at 0  
  
}
```

המטרה היא אתחול כל המשתנים למצב התחלתי כאשר מתחיל משחק חדש, לצורך מעקב תקין אחר התקדמות המשתמש.

פונקציות גישה

פונקציות קריאה בלבד המאפשרות שליפה של נתוני הניקוד:

```
int get_correct_answers() const;
```

```
int get_wrong_answers() const;
```

```
int get_total_images() const;
```

```
bool is_game_complete() const;
```

מטרתן היא לספק גישה מבוקרת לערכי הניקוד מבלי לשנות אותם ישירות מתוך קוד חיצוני.

אירועי עדכון ניקוד

הניקוד מתעדכן בשלושה מצבים שונים במהלך המשחק:

1. תשובה נכונה:

```
void Game::handle_success() {
```

```
    correct_answers_m++;
```

```
}
```

2. תשובה שגויה:

```
void Game::handle_failure() {
```

```
    wrong_answers_m++;
```

```
}
```

3. סוף הזמן (נחשב גם הוא לתשובה שגויה על מנת לעודד את המשתמש לענות במהירות):

```
static void on_timer_timeout(lv_anim_t * a) {  
  
    increment_wrong_answers();  
  
}
```

זיהוי סיום המשחק

המערכת מזהה כי המשחק הסתיים כאשר כל התמונות הוצגו לפחות פעם אחת:

```
void iterate_image() {  
  
    images_completed_m++;  
  
    if (images_completed_m >= image_name_list_m.size()) {  
  
        game_loop_complete_m = true; // Game is done!  
  
    }  
  
}
```

לאחר כל תשובה (נכונה, שגויה, או סיום הזמן), מתבצעת בדיקה:

```
if (is_game_complete()) {  
  
    show_results_screen();  
  
    return;  
  
}
```

המטרה היא לוודא שסיום המשחק מתרחש רק לאחר שכל התמונות הוצגו, ולהפעיל את מסך התוצאות בזמן הנכון.

הצגת מסך התוצאות

בעת סיום המשחק, מוצגות התוצאות הסופיות של המשתמש:

```
void show_results_screen() {  
  
    int correct = global_game->get_correct_answers();  
  
    int wrong = global_game->get_wrong_answers();  
  
    int total = global_game->get_total_images();  
  
  
    char score_text[50];  
  
    snprintf(score_text, sizeof(score_text), "%s: %d/%d", get_text_score(), correct, total);  
  
}
```

רכיבי ממשק משתמש: (UI Elements)

- כותרת: "המשחק הסתיים! / Game Complete!".
- הצגת מספר תשובות נכונות (בירוק), שגיאות (באדום), והניקוד הכולל (בלבן).
- כפתור "שחק שוב / Play Again" לאתחול המשחק מחדש.

אתחול המשחק מחדש (Game Reset)

בעת לחיצה על "שחק שוב", המערכת מאפסת את כל המונים ומחזירה את המשחק לתמונה הראשונה:

```
void play_again_callback(lv_event_t *e) {  
  
    global_game->reset_game(); // Zeros all counters  
  
}
```

```
void reset_game() {  
  
    correct_answers_m = 0;  
  
    wrong_answers_m = 0;  
  
    images_completed_m = 0;  
  
    game_loop_complete_m = false;  
  
    image_pos_m = 0; // Back to first image  
  
}
```

המטרה היא לאפשר חוויית משחק מחזורית - שחקן יכול להתחיל משחק חדש כשהמערכת חוזרת למצב התחלתי נקי.